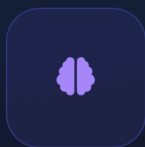


Sprint 3

Agentic AI Python Tutor — Neue Features



AGENT-GEDÄCHTNIS



CODE-REVIEW CHAIN



LERNPLAN-GENERATOR

AGENDA



SLIDE 3 — JOURNEY

Lisa's Lernstunde

Eine Studentin vom Login bis zum personalisierten Lernplan — alle neuen Features in einer echten Session.



SLIDE 4 — GEDÄCHTNIS

Kein Neustart mehr

Der Agent erinnert sich session-übergreifend — *"Letzte Woche haben wir Loops besprochen..."* ohne dass Lisa es wiederholen muss.



SLIDE 5 — REVIEW

3-Ebenen-Feedback

Syntax ● · Stil ● · Architektur ● — jede Ebene separat, damit Lisa weiß, *was zuerst zu fixen ist.*



SLIDE 6 — LERNPLAN

Personalisiert

Plan basiert auf echten Skill-Scores — schwache Themen zuerst, *nicht eine generische To-do-Liste.*



SLIDE 7 — AUSBLICK

Nächster Sprint

Adaptiver Hinweis-Dialog · Fehlermeldungen auf Deutsch · Code-Umschreibungs-Tool — Sprint 4 Planung.



SLIDE 8 — DEMO

Live Demo

System in Aktion — alle neuen Sprint-3-Features live zeigen.

USER JOURNEY

Lisa's Lernstunde — Eine Session von Anfang bis Ende



Lisa — Python-Anfängerin, 2. Semester. Sie hat letzte Woche Loops gelernt und will heute ihren ersten Code reviewen lassen.

**Dashboard**

Login → zwei klare
Optionen: *KI Tutor*
oder *Python Kurs*

`DashboardView.tsx`**Tutor startet**

"Hallo! 🖐️ *Ich bin dein
KI-Tutor!*" — Roboter
begrüßt sie mit
Animation

`TutorTransition.tsx`**Tutor erinnert sich**

Agent: "*Letzte Woche
haben wir Loops
besprochen — weiter
damit?*"

`memory_service.py`**Code Review**

Ihr Loop-Code
bekommt **3 Ebenen**
Feedback: Syntax 🔴,
Stil 🟡, Architektur 🔵

`RunnableSequence`**Lernplan**

Personalisierter
Wochenplan:
*schwache Skills
zuerst*, max. 3 Skills
pro Woche

`/learning-plan`**Nächste Woche**

Lisa kommt zurück —
Tutor weiß noch alles.
Kein Neustart.

`memory_summary in DB`

WIE FUNKTIONIERT'S? — AGENT-GEDÄCHTNIS

memory_service.py — Eigene Memory-Implementierung

⚡ BEIM SEITENAUFBRUF — SUMMARY SOFORT SICHTBAR

Schon *bevor* Lisa etwas schreibt: `GET /tutor/memory` lädt den Summary aus der DB und zeigt ihn als 🧠-Karte oben im Chat. Kein Warten, kein Klick — Lisa sieht sofort, was der Tutor über sie weiß.

⚡ LLM-AUFRUF 1 — ANTWORT MIT KONTEXT

Erst wenn Lisa eine Nachricht *schickt*: `load_memory()` packt den Summary als Kontext in den System-Prompt. `run_chat()` antwortet — mit Wissen über vergangene Sessions.

⚡ LLM-AUFRUF 2 — MEMORY AKTUALISIEREN

Nach der Antwort ruft `update_memory()` das LLM ein zweites Mal auf: alter Summary plus neue Nachrichten rein — neuer Summary raus. Format: **max. 2 Sätze · max. 40 Wörter · nur Schlüsselwörter**. Direkt in DB gespeichert.

✗ OHNE GEDÄCHTNIS

Lisa schreibt: *"Ich hab Probleme mit Loops"*

"Hallo! Schön, dass du hier bist. Was möchtest du heute lernen? Bist du Anfänger oder hast du schon Erfahrung?"

✓ MIT GEDÄCHTNIS

Lisa schreibt: *"Ich hab Probleme mit Loops"*

"Letzte Woche hatten wir schon For-Schleifen angeschaut — da hattest du Schwierigkeiten mit dem Index. Wollen wir genau da weitermachen?"

WIE FUNKTIONIERT'S? — CODE-REVIEW CHAIN

code_review_chain.py — 3 Schritte · 3 LLM-Aufrufe

Lisa schickt Code rein — eine `RunnableSequence` mit **drei Schritten und drei separaten LLM-Aufrufen** läuft durch. Jeder Schritt hat einen eigenen System-Prompt und fokussiert auf genau eine Dimension. Verfügbar **direkt in jeder Übungskarte** per "Code Review"-Button.

● SCHRITT 1 — SYNTAX

Nur echte Fehler: fehlende Doppelpunkte, falsche Einrückung, undefinierte Variablen — mit Zeilennummer und Schweregrad `error/warning`.

● SCHRITT 2 — STIL / PEP8

Anderer Prompt — PEP8, Naming (snake_case), Zeilenlänge, Magic Numbers, fehlende Docstrings. Bekommt den Syntax-Summary als Kontext aus Schritt 1.

● SCHRITT 3 — BEST PRACTICES

✗ Schlecht

```
if is_valid == True:
```

✓ Besser

```
if is_valid:
```

WIE FUNKTIONIERT'S? — LERNPLAN-GENERATOR

learning_plan_tool.py — Intelligent sortiert, gecacht, mit Lernschritten

 SORTIERREIHENFOLGE

1. Freigeschaltete Skills mit *hohem Score zuerst* — fast fertig, leicht auf 100% zu bringen. 2. Freigeschaltete Skills mit niedrigem Score. 3. Gesperrte Skills *immer am Ende* — nie in Woche 1. Max. 3 Skills/Woche, max. 4 Wochen.

 LERNSCHRITTE PRO SKILL

Jeder Skill wird in **2–3 konkrete Schritte** aufgeteilt — deterministisch nach Score: Score < 30 → Wiederholen + Grundübungen + Schwerpunkt · Score 30–59 → Wiederholen + Üben · Score 60–79 → Auffrischen + Lücken schließen. Mit Zeitschätzung pro Schritt.

 GESPEICHERT IM BROWSER

Der generierte Plan wird im `localStorage` gecacht (`ki_tutor_plan_<userId>`) — beim nächsten Öffnen erscheint er sofort, kein LLM-Call. Datum der letzten Generierung sichtbar, Refresh-Button für neuen Plan nach Fortschritt. Cache-Ablauf nach **4 Wochen**.

NÄCHSTER SPRINT

Was kommt als nächstes?



Adaptiver Hinweis-Dialog

Der Tutor erkennt, wann ein Lernender nicht weiterkommt, und bietet kontextbezogene Hinweise in drei Stufen an — ohne die Lösung direkt zu verraten.

☰ Stufe 1 · Stufe 2 · Lösung

Hint-on-Demand · progressiv · LLM-generiert

SPRINT 4



Fehler-Erklärung auf Deutsch

Python-Fehlermeldungen werden automatisch erkannt, übersetzt und auf Deutsch verständlich erklärt — ideal für deutschsprachige Lernende.

🔍 Error-Detection · DE-Übersetzung · Beispiel

Traceback-Parsing · LLM-Erklärung · kontextsensitiv

SPRINT 4



Inline Thema-Erklärung

Direkt in der Übungsansicht: ein aufklappbares Panel erklärt das Thema per AI — ohne zur Tutor-Seite zu wechseln. Übungskontext bleibt erhalten.

▼ Aufklappbar · gecacht · kein Seitenwechsel

Inline-Panel · AI-generiert · level-spezifisch

SPRINT 4

Live Demo